

INSTITUTO FEDERAL CATARINENSE
BACHARELADO DE CIÊNCIA DA COMPUTAÇÃO

C++
Guia de Estudo Completo

André Vitor Bastos de Macêdo

Blumenau - SC
2026

Sumário

1	Tipos e Variáveis	4
1.1	Tipos Primitivos e Modificadores	4
1.2	Variáveis, Constantes e constexpr	4
1.2.1	Inicialização Uniforme	4
1.2.2	A palavra-chave auto	4
1.2.3	Structured Bindings (C++17)	4
1.3	Tipos Compostos	5
1.3.1	Arrays	5
1.3.2	Enums	5
1.4	Strings	5
1.5	Entrada e Saída	5
2	Operadores e Controle de Fluxo	5
2.1	Operadores	5
2.1.1	Aritméticos	5
2.1.2	Lógicos	5
2.1.3	Bitwise	5
2.2	Estruturas Condicionais	5
2.3	Laços de Repetição	5
2.4	Tratamento de Exceções	5
3	Funções e Organização de Código	5
3.1	Definição, Escopo e Visibilidade	5
3.2	Passagem de Parâmetros	5
3.2.1	Valor	5
3.2.2	Referência	5
3.2.3	Ponteiro	5
3.3	Retornos de Função e std::optional	5
3.4	Recursividade	6
3.5	Sobrecarga de Funções e Inline	6
3.6	Funções Lambda e Callbacks	6
4	O Modelo de Memória do C++	7
4.1	Stack vs Heap	7
4.2	Alocação Estática, Automática e Dinâmica	7
4.3	Ciclo de Vida de Objetos e Escopo	7
4.4	Fragmentação e Performance de Cache	7
5	Ponteiros e Referências	7
5.1	Ponteiros Simples e Aritmética de Ponteiros	7
5.2	Referências	7
5.3	Ponteiros para Funções e Membros	7
5.4	Ponteiros Nulos e nullptr	7
5.5	Type Casting	7
6	Gerenciamento de Recursos	7
6.1	O Conceito de RAII	7
6.2	Gerenciamento Manual	7

6.3	Ponteiros Inteligentes	7
6.4	Movimentação de Recursos (Move Semantics)	8
7	Orientação a Objetos	9
7.1	Encapsulamento e Modificadores de Acesso	9
7.2	Construtores, Destrutores e a “Rule of Five”	9
7.3	Herança e Polimorfismo	9
7.4	Funções Virtuais e Classes Abstratas	9
7.5	Sobrecarga de Operadores	9
8	Programação Genérica	9
8.1	Templates de Função	9
8.2	Templates de Classe	9
8.3	Especialização de Templates	10
9	STL (Standard Template Library)	10
9.1	Containers	10
9.2	Iteradores e sua importância	10
9.3	Algoritmos e Ranges (C++20)	10
9.4	Functors e Predicados	11

1 Tipos e Variáveis

1.1 Tipos Primitivos e Modificadores

1.2 Variáveis, Constantes e constexpr

1.2.1 Inicialização Uniforme

Antes do C++11, inicializar variáveis era uma confusão: usávamos parênteses (), sinal de igual = ou listas de inicialização para agregados. A inicialização com chaves {} veio para unificar isso e resolver problemas clássicos, como o “Most Vexing Parse” (quando o compilador confunde a declaração de uma variável com a declaração de uma função).

```
int temp_A = 36.6; // Estilo antigo
int temp_B{36.6}; // Inicialização Uniforme (C++ Moderno)
```

Aviso de Segurança: Narrowing Conversions

No estilo antigo com =, o compilador realiza uma conversão implícita (truncando o valor decimal para caber no int). Isso acontece silenciosamente e pode gerar bugs. Com a Inicialização Uniforme {}, o compilador é rigoroso: se o valor não couber perfeitamente no tipo (como um double em um int), ele emitirá um erro, impedindo o código de compilar com um valor corrompido.

1.2.2 A palavra-chave auto

O C++ Moderno introduziu a palavra-chave auto para dedução de tipos pelo compilador. Porém, há uma armadilha ao misturar auto com chaves {}:

```
auto var1 = 10; // Deduzido como int
auto var2{10}; // Deduzido como int
auto var3 = {10}; // Cuidado: deduzido como std::initializer_list<int>!
```

O tipo std::initializer_list<T> serve como uma “ponte” temporária para containers, capaz de ser transicionada para diferentes estruturas de dados.

1.2.3 Structured Bindings (C++17)

Permite extrair múltiplos retornos ou desempacotar estruturas diretamente em variáveis individuais, de forma muito limpa.

```
struct Config { int id; float sens; };

Config c{10, 0.99f};
auto [id, sens] = c; // 'id' recebe 10, 'sens' recebe 0.99
```

O número de identificadores nos colchetes deve ser exatamente igual ao número de membros não estáticos da struct.

1.3 Tipos Compostos

1.3.1 Arrays

1.3.2 Enums

1.4 Strings

1.5 Entrada e Saída

2 Operadores e Controle de Fluxo

2.1 Operadores

2.1.1 Aritméticos

2.1.2 Lógicos

2.1.3 Bitwise

2.2 Estruturas Condicionais

2.3 Laços de Repetição

2.4 Tratamento de Exceções

3 Funções e Organização de Código

3.1 Definição, Escopo e Visibilidade

3.2 Passagem de Parâmetros

3.2.1 Valor

3.2.2 Referência

3.2.3 Ponteiro

3.3 Retornos de Função e `std::optional`

No C++ antigo, ao falhar uma busca, retornávamos um ponteiro nulo (`nullptr`) ou um valor sentinela (como `-1`). Com o C++17, ganhamos o `std::optional<T>`. Ele encapsula a possibilidade de “ter um valor” ou “estar vazio”.

```
#include <optional>
#include <vector>

struct Config { int id; float sensitivity; };

std::optional<Config> buscar_por_id(const std::vector<Config>& lista,
```

```
int id_procurado) {
    for (const auto& item : lista) {
        if (item.id == id_procurado) return item;
    }
    return std::nullopt; // Retorna "vazio"
}

// Uso:
auto resultado = buscar_por_id(configs, 2);
if (resultado) { // Pode ser checado como um booleano
    std::cout << resultado->sensitivity; // Acesso via operador seta
}
```

O `std::optional` se comporta intuitivamente como um ponteiro inteligente, mas com a segurança de que reside no *stack* e não no *heap*.

3.4 Recursividade

3.5 Sobrecarga de Funções e Inline

3.6 Funções Lambda e Callbacks

Funções lambda são funções anônimas que podemos definir dentro do próprio código, frequentemente usadas com algoritmos da STL.

Anatomia Básica: [captura] (parâmetros) { corpo; }

```
#include <algorithm>
#include <vector>

auto list = {10, 20, 30, 40};
auto limit = 25;

// Passando um lambda para o std::for_each
std::for_each(list.begin(), list.end(), [limit](int value) {
    if (value > limit) {
        std::cout << value << std::endl;
    }
});
```

Tipos de Captura:

- `[]`: Captura nada.
- `[limit]`: Captura por valor. Faz uma cópia da variável local no momento da criação da lambda.
- `[&limit]`: Captura por referência. Acessa a variável original.
- `[=]` / `[&]`: Captura tudo o que for necessário do escopo externo por valor ou por referência, respectivamente.

4 O Modelo de Memória do C++

4.1 Stack vs Heap

4.2 Alocação Estática, Automática e Dinâmica

4.3 Ciclo de Vida de Objetos e Escopo

4.4 Fragmentação e Performance de Cache

5 Ponteiros e Referências

5.1 Ponteiros Simples e Aritmética de Ponteiros

5.2 Referências

5.3 Ponteiros para Funções e Membros

5.4 Ponteiros Nulos e nullptr

5.5 Type Casting

6 Gerenciamento de Recursos

6.1 O Conceito de RAII

Resource Acquisition Is Initialization (RAII) é a base do gerenciamento seguro no C++.

Regra de Ouro do C++ Moderno:

Nunca use `new` ou `delete` manualmente no seu código de alto nível. Deixe que as abstrações (Smart Pointers e Containers) gerenciem isso para você.

6.2 Gerenciamento Manual

6.3 Ponteiros Inteligentes

Os Smart Pointers automatizam o gerenciamento de memória baseando-se no escopo (RAII). Os principais são:

1. `std::unique_ptr<T>`: Propriedade exclusiva. Ninguém mais pode apontar para o recurso. Quando sai de escopo, o recurso é destruído automaticamente. É “grátis” em termos de performance.

```
auto motor = std::make_unique<Motor>();  
// auto motor2 = motor;  
// ERRO! Não pode ser copiado.  
auto motor2 = std::move(motor);  
// Correto: a posse foi transferida. motor agora é nullptr.
```

2. `std::shared_ptr<T>`: Propriedade compartilhada. Mantém um bloco de controle (Control Block) com um contador de referências. O recurso só morre quando o último ponteiro for destruído (contador chega a zero).

O problema da Referência Circular: Se um objeto A tem um `shared_ptr` para B, e B tem um `shared_ptr` para A, nenhum dos dois será destruído, causando um *Memory Leak*.

3. `std::weak_ptr<T>`: A solução para a referência circular. Ele observa um `shared_ptr`, mas não incrementa o contador de referências. Para acessar o valor, você deve “promovê-lo” temporariamente usando `lock()`, garantindo *Thread Safety*.

6.4 Movimentação de Recursos (Move Semantics)

Para entender a movimentação, precisamos diferenciar:

- **Lvalue:** Tem nome e endereço de memória fixo (ex: uma variável x).
- **Rvalue:** É temporário, prestes a evaporar (ex: resultado de `10 + 20` ou retorno de uma função).

O **Move Constructor** permite “roubar” os recursos de um Rvalue (objeto temporário) em vez de fazer uma cópia custosa.

```
class Buffer {
    int* data;
    size_t size;
public:
    // Construtor de Cópia Tradicional (Custa caro)
    Buffer(const Buffer& other) { /* Aloca nova memória e copia os
dados */ }

    // Construtor de Movimento (Rápido)
    Buffer(Buffer&& other) noexcept {
        this->size = other.size;
        this->data = other.data; // "Rouba" o ponteiro

        other.size = 0;
        other.data = nullptr;    // Fundamental: anula o original para
evitar Double Free
    }
};
```

Ao definir `other.data = nullptr`, garantimos que quando o objeto temporário for destruído, o seu destrutor chamará `delete[] nullptr` (o que é seguro e não faz nada).

O Idioma Copy-and-Swap: Uma forma mais elegante e segura contra exceções de implementar a cópia e o movimento é usando `std::swap`:

```
Buffer(Buffer&& other) noexcept : data(nullptr), size(0) {
    std::swap(this->data, other.data);
    std::swap(this->size, other.size);
}
```



```
Buffer& operator=(Buffer other) noexcept { // Recebe por valor (cópia ou movido)
    std::swap(this->data, other.data);
    std::swap(this->size, other.size);
    return *this;
}
```

7 Orientação a Objetos

7.1 Encapsulamento e Modificadores de Acesso

7.2 Construtores, Destrutores e a “Rule of Five”

7.3 Herança e Polimorfismo

7.4 Funções Virtuais e Classes Abstratas

7.5 Sobrecarga de Operadores

8 Programação Genérica

8.1 Templates de Função

O Template funciona como um molde. Você escreve uma receita para o compilador, e o código real só é “fabricado” quando você o utiliza com um tipo específico.

```
template <typename T>
T somar(T a, T b) {
    return a + b;
}

auto r1 = somar<int>(5, 10);           // Compilador gera a versão 'int'
auto r2 = somar<double>(5.5, 2.1);    // Compilador gera a versão 'double'
auto r3 = somar(1, 2);                // O compilador deduz que é 'int'
```

8.2 Templates de Classe

A mesma lógica se aplica a classes, como na criação de estruturas de dados genéricas.

```
template<typename T>
class Buffer {
    T* data;
    size_t size;
public:
    Buffer(size_t s);
    ~Buffer();
}
```

```
// ... construtores, begin(), end() ...
};

template<typename T>
Buffer<T>::Buffer(size_t s) : size(s), data(new T[s]) {}
```

Atenção à Organização de Arquivos:

Diferente de classes normais, você **não pode** colocar a declaração no .hpp e a implementação no .cpp. Como o compilador precisa gerar o código sob demanda, ele precisa enxergar a implementação completa. Implemente templates inteiramente no arquivo de cabeçalho (.hpp ou .h).

8.3 Especialização de Templates

9 STL (Standard Template Library)

9.1 Containers

9.2 Iteradores e sua importância

Para que uma classe customizada funcione com os for-loops baseados em range (for (auto x : buffer)), ela deve implementar as funções begin() e end().

```
template<typename T> T* Buffer<T>::begin() { return data; }
template<typename T> T* Buffer<T>::end() { return data + size; }
```

9.3 Algoritmos e Ranges (C++20)

No C++20, um *Range* é essencialmente qualquer coisa que possua um begin() e um end(). Isso simplificou drasticamente o uso dos algoritmos da STL.

```
// Como era antes (C++11/14/17):
std::reverse(buffer.begin(), buffer.end());

// Como é agora (C++20 Ranges):
std::ranges::reverse(buffer);
```

Para ordenar estruturas complexas (como um vetor de Jogador), temos duas abordagens principais: usando funções Lambda ou sobrecarregando o operator<.

```
struct Jogador {
    std::string nome;
    int pontuacao;

    // Caminho 1: Implementando o operator<
    bool operator<(const Jogador& other) const {
```

```

        return pontuacao < other.pontuacao;
    }
};

std::vector<Jogador> jogadores = {"A", 200}, {"B", 150}, {"C", 180}};

// Usando o operator< (deduzido por padrão)
// O std::ranges pede a struct, uma projeção vazia {}, e o membro a
comparar
std::ranges::sort(jogadores, {}, &Jogador::pontuacao);

// Caminho 2: Usando Lambda (mais flexível para contextos locais)
std::ranges::sort(jogadores, [](const Jogador& a, const Jogador& b) {
    return a.pontuacao < b.pontuacao;
});

```

9.4 Functors e Predicados